

Pending Agent Architecture & Implementation Report

Comprehensive Technical Guide: Components, Modifications, State Machines, LLM Pipeline & Flaw Resolutions
Incident AI Platform • Date: September 8, 2026 • Status: Production Verified

Executive Summary

The **Pending Agent** is an autonomous ITSM lifecycle agent designed to handle ServiceNow tickets entering **On Hold** / **Pending** states. It continuously monitors engineer work notes, analyzes what information or action is required from the caller using Groq LLM, manages an automatic 3-tier reminder cycle, generates simulated notification emails, preserves historical audit trails in PostgreSQL, and enforces clean state consistency across UI and backend services.

1. Created Files & Component Responsibilities

The Pending Agent module was built using clean, decoupled modular architecture mirroring the platform's Triage and Acknowledgement agents under `backend/app/modules/agents/pending/` :

File Path	Component	Purpose & Responsibility
<code>models/pending_cycle.py</code>	SQLAlchemy Model	Defines the <code>PendingCycle</code> ORM model and PostgreSQL enum <code>PendingCycleStatus</code> (<code>ACTIVE</code> , <code>COMPLETED</code> , <code>CANCELLED</code>). Stores cycle ID, incident ID/number, reminder count (1-3), max reminders, source type, and <code>next_reminder_at</code> timestamp.
<code>schemas.py</code>	Pydantic Schemas	Data transfer objects: <code>PendingWorkNoteAnalysis</code> (LLM output parser), <code>PendingResult</code> (execution telemetry), <code>PendingCycleResponse</code> (API serialization), and <code>PendingTriggerRequest</code> .
<code>repository.py</code>	Data Access Layer	<code>PendingCycleRepository</code> handles transactional database operations: <code>get_active_cycle()</code> , <code>get_latest_cycle()</code> , <code>create_cycle()</code> , <code>increment_reminder()</code> , <code>complete_cycle()</code> , and <code>cancel_cycle()</code> with explicit <code>commit()</code> and <code>refresh()</code> calls.
<code>work_note_analyzer.py</code>	AI Intelligence Layer	<code>PendingWorkNoteAnalyzer</code> calls the Groq LLM service. Extracts specific caller action requirements, error details, screenshot requests, and employee credentials from recent work notes and ticket context.
<code>reminder_renderer.py</code>	Template Renderer	<code>PendingReminderRenderer</code> formats plain-text and HTML emails using Jinja2. Adapts tone and urgency across Reminder 1 (friendly check-in), Reminder 2 (follow-up), and Reminder 3 (final notice before auto-closure).
<code>templates/standard_pending_reminder.html</code>	Email Template	Responsive HTML email template with dynamic placeholders for <code>ticket_number</code> , <code>caller_name</code> , <code>assigned_engineer_name</code> , <code>awaiting_detail</code> , reminder count, and ServiceNow portal links.
<code>agent.py</code>	BaseAgent Concrete Class	<code>PendingAgent</code> implements the standard <code>BaseAgent</code> contract, taking <code>AIContext</code> , parsing work notes from ticket history, and returning standardized <code>AgentResponse</code> .
<code>service.py</code>	Business Orchestrator	<code>PendingService</code> coordinates the complete lifecycle: idempotency checks (cycle preservation), multi-note context extraction, Groq LLM analysis, cycle advancement, work note generation, and state enforcement.
<code>api/v1/pending.py</code>	API Router	FastAPI routes: <code>POST /v1/pending/{incident_id}</code> for automated/scheduled execution, and <code>GET /v1/pending/{incident_id}/cycle</code> for frontend status badges.
<code>tests/unit/test_pending_agent.py</code>	Automated Test Suite	5 unit tests verifying Reminder 1 rendering, Final Reminder 3 rendering, Acknowledgement Agent fast path, idempotency cycle preservation, and multi-note context analysis.
<code>PENDING_AGENT_SIMPLE_WORKFLOW.md</code>	Specification Doc	Architectural blueprint specifying trigger points, database schema, prompt templates, and lifecycle boundaries.

2. Modifications to Existing Files

To seamlessly integrate the Pending Agent without regressions, specific surgical updates were made across existing modules:

Modified File	Changes Made	Reason & Impact
backend/app/main.py	Imported and mounted <code>pending_router</code> via <code>app.include_router(pending_router, prefix="/v1")</code> .	Exposes Pending Agent endpoints under the platform's v1 API schema.
backend/app/api/v1/incidents.py	1. Removed <code>background_tasks.add_task(_auto_triage_force, incident_id)</code> from <code>PUT /v1/incidents/{incident_id}</code> . 2. Removed the dead <code>_auto_triage_force()</code> function.	Critical Fix: Prevents manual incident updates from being hijacked by background triage and acknowledgement agents which were forcing active tickets back into On Hold.
backend/app/modules/incidents/service.py	1. Added <code>self.db = db</code> to <code>IncidentService.__init__</code> . 2. Changed manual edit work note author to <code>WorkNoteSourceType.ENGINEER</code> and engineer name. 3. Hooked <code>PendingService.process_pending_transition()</code> on state transition to <code>on_hold</code> . 4. Hooked <code>cancel_cycle()</code> on state transitions away from <code>on_hold</code> (e.g. to <code>in_progress</code>).	1. Fixed internal crash (missing attribute <code>'db'</code>). 2. Ensures work notes attribute changes to the Engineer instead of generic 'User'. 3. Automates Pending Agent triggering. 4. Cancels active pending cycles when an engineer resumes work.
backend/app/modules/agents/acknowledgement/service.py	Added automatic invocation of <code>PendingService.process_pending_transition()</code> when Acknowledgement Agent transitions a ticket to <code>on_hold</code> .	Initiates Reminder 1 automatically when Acknowledgement Agent routes users to raise RITM forms.
frontend/src/api/client.js	Added <code>pendingApi = { getCycle: (id) => api.get(`/pending/\${id}/cycle`) }</code> .	Enables frontend UI to query pending cycle state.
frontend/src/pages/IncidentDetail.jsx	1. Removed automatic <code>trriageApi.run()</code> hook on update success. 2. Added query for <code>pendingApi.getCycle(id)</code> . 3. Added dynamic badge <code>Reminder X/3 Active</code> under Classification card.	Prevents frontend from re-triaging upon editing; displays cycle status without adding unwanted manual buttons.

3. End-to-End Workflow & State Machine

The diagram below outlines the full lifecycle of an incident moving through the Pending Agent system:

Step 1: State Change Trigger

Incident state updates to **On Hold** (via manual engineer edit in UI or automatic Acknowledgement Agent classification).



Step 2: Idempotency Gate (Cycle Preservation)

PendingService checks for an existing **ACTIVE** cycle in PostgreSQL.

- If an active cycle exists and reminder is not forced: **Preserves cycle**, retains timer, returns **CYCLE_PRESERVED**.
- If no active cycle exists: Advances to note analysis.



Step 3: Multi-Note Context & Substantive Fallback

Fetches top 2 work notes (Note 0 + Note 1). If both are state transition messages (**State changed to: ...**), the query looks back for the most recent substantive note (e.g. manual engineer inquiry or system note) plus incident title and description.



Step 4: Groq LLM Dynamic Extraction

Work notes and context are sent to Groq API (**POST https://api.groq.com/openai/v1/chat/completions**). LLM parses requirements and returns: **{"is_caller_action_required": true, "reasoning": "...", "confidence": 0.98}**.



Step 5: Database Cycle Creation / Advancement

A new **PendingCycle** record is inserted (**status: ACTIVE** , **reminder_count: 1/3** , **next_reminder_at: +24h**). All transactions are explicitly committed and refreshed.



Step 6: Work Note Audit Trail Generation

The full plain-text reminder email is rendered with exact details needed from caller. A structured work note (**source_type: PENDING_AGENT** , **action_type: SEND_REMINDER**) is appended containing the full email text and cycle ID.

4. Flaws Discovered, Erased & Solved

During development and live verification, several intricate architectural and concurrency issues were identified and permanently resolved:

Flaw 1: Unwanted Auto-Triage on Incident Updates (Reverting Active to On Hold)

- **Symptom:** When an engineer clicked Edit and activated a ticket to `In Progress`, the UI refreshed and showed the ticket reverted to `On Hold`.
- **Root Cause:** `backend/app/api/v1/incidents.py` had an automatic hook in `update_incident`:

```
if updated.state in ('new', 'in_progress'):
    background_tasks.add_task(_auto_triage_force, incident_id)
```

This background task executed `TriageAgent`, which in turn called `AcknowledgementAgent`. The Acknowledgement Agent re-classified the description as a Salesforce request, changed state back to `On Hold`, and initiated a new pending cycle!

- **Solution:** Completely removed `_auto_triage_force` and the background task from `PUT /v1/incidents/{incident_id}`. Triage now only triggers on initial creation or when explicitly run by the user. Manual edits permanently persist.

Flaw 2: Missing `self.db` Attribute on `IncidentService`

- **Symptom:** Moving an incident from `On Hold` to `In Progress` logged an unhandled exception: `'IncidentService' object has no attribute 'db'`, and the pending cycle remained `ACTIVE`.
- **Root Cause:** In `IncidentService.__init__`, the constructor initialized `self.repo = IncidentRepository(db)` but failed to store `self.db = db`. When `update_incident` called `PendingCycleRepository(self.db)` to cancel active cycles, Python raised an `AttributeError`.
- **Solution:** Added `self.db = db` to `IncidentService.__init__`. Cycle cancellation now executes cleanly.

Flaw 3: SQLAlchemy Session Flush vs. Commit in `PendingCycleRepository`

- **Symptom:** Cycle creations and state updates were lost upon HTTP request completion; database tables showed stale statuses.
- **Root Cause:** `PendingCycleRepository` was executing `await self.db.flush()` instead of `await self.db.commit()`. FastAPI's database session dependency closes sessions upon response delivery, triggering an implicit rollback of uncommitted flushes.
- **Solution:** Replaced all instances of `flush()` with `await self.db.commit()` and `await self.db.refresh(cycle)` across all repository methods.

Flaw 4: Consecutive State Change Notes Causing Silent Reminder Discard

- **Symptom:** Manually setting a ticket to `On Hold` did not trigger the Pending Agent; no reminder note was generated and the pending cycle was missing.
- **Root Cause:** In the work notes history, the two newest entries were:
 1. `State changed to: on_hold...`
 2. `State changed to: in_progress...`

Because neither note asked a question, Groq LLM returned `is_caller_action_required: false`. A guard in `service.py` then took a `DISCARDED` branch and returned early without generating a reminder!

- **Solution:**
 1. Updated `PendingWorkNoteAnalyzer` prompt to always enforce `is_caller_action_required: true` for tickets placed in On Hold.
 2. Enhanced note selection: if the top two notes are state changes, the algorithm looks back for the most recent substantive note (e.g. engineer question or acknowledgement text).
 3. Removed the discard branch in `PendingService` so reminder generation always executes on On Hold transitions.

Flaw 5: Work Note Attribution Mismatch

- **Symptom:** State change notes added during manual edits were attributed to `source_type: USER` and name `"User"`.
- **Root Cause:** `IncidentService.update_incident` was defaulting source to `USER`.
- **Solution:** Updated `update_incident` to attribute notes to `WorkNoteSourceType.ENGINEER` and the assigned engineer's name (or `"Engineer"` if unassigned).

5. Proof of Real LLM Execution (No Hardcoding)

To verify that work notes are dynamically processed through Groq's LLM rather than pattern-matched or hardcoded, a live audit test was executed with unique, non-standard text:

Test Input Work Note (Posted by Engineer):

Please upload your error log file named error_dump_9942.log and send your laptop MAC address to proceed.

Real Server Log Output (Groq API Call):

```
2026-09-08 21:57:13,423 - app.modules.agents.pending.work_note_analyzer - INFO -
[WorkNoteAnalyzer] === SENDING TO GROQ LLM ===
INCIDENT CONTEXT:
Ticket: INC0000070 | Title: Need access to Salesforce GTS EANZ and profile permission
RECENT WORK NOTES (LATEST & BELOW IT):
Author / Source: Suresh Raina
[Latest Note - Suresh Raina (ENGINEER)]: State changed to: on_hold.
[Previous Note (Below Latest) - Suresh Raina (ENGINEER)]:
Please upload your error log file named error_dump_9942.log and send your laptop MAC address to proceed.
=====
2026-09-08 21:57:15,627 - httpx - INFO - HTTP Request: POST https://api.groq.com/openai/v1/chat/completions "HTTP/1.1 200 OK"
2026-09-08 21:57:15,634 - app.modules.agents.pending.work_note_analyzer - INFO -
[WorkNoteAnalyzer] === GROQ LLM RESPONSE ===
{
  "is_caller_action_required": true,
  "reasoning": "Caller needs to upload the error log file named error_dump_9942.log and provide the laptop MAC address.",
  "confidence": 0.98
}
=====
```

Resulting Structured Work Note:

```
Pending Agent: Reminder 1 of 3 sent to user.
• Cycle ID: b12ed96b-8a45-4de4-b388-8958f941965c
• Awaiting from User: Caller needs to upload the error log file named error_dump_9942.log and provide the laptop MAC address.
• Follow-up Status: Awaiting User Response
• State: Maintained in Pending

===== EMAIL SENT TO USER =====
Subject: Follow-up Reminder (1/3) - Ticket INC0000070
...
• Details Needed from You: Caller needs to upload the error log file named error_dump_9942.log and provide the laptop MAC address
...
```

6. Verification & Test Suite Summary

The implementation has been thoroughly verified across unit tests, database queries, and live browser flows:

- **Unit Test Suite:** `pytest app/tests/ -v` → **46 passed** (100% test pass rate).
- **Database Consistency:** Verified records in PostgreSQL table `pending_cycles` transition cleanly across `ACTIVE`, `CANCELLED`, and `COMPLETED`.
- **UI Badge:** `Reminder X/3 Active` displays dynamically under Classification card and disappears when ticket is activated or cancelled.
- **No UI Button:** The Pending Agent triggers strictly through state events without cluttering the UI with manual action buttons.